Escuela Técnica N°1 Otto Krause



Laboratorio de Programación Orientada a Objetos

Guión - Ejercitación 16

Alumnos: Toledano Jazmin
Pagni Octavio

Curso: 5°2 Computación

Prof: Aaron Sebastian Serrano

Ciclo Lectivo 2026

Consignas:

1. Idea del juego en 3-4 líneas: de qué se trata, qué hace el jugador.

Decidimos utilizar la idea de nuestro proyecto cuatrimestral de proyecto informático, un juego ambientado en el buffet de la escuela.

En este juego, el jugador tiene que realizar diferentes tareas, atender a los clientes que llegan al buffet y administrar los objetos que posee en su inventario.

El juego se desarrollará completamente mediante líneas de texto plano en una consola, donde el jugador deberá seleccionar diferentes opciones para realizar las acciones disponibles.

ORDEN DE LAS CARPETAS Y SUS RESPECTIVAS CLASES:

MODEL
│
├── Entidad
├── Personaje
├── Jugador
├── NPC
├── Cliente
├── Objeto
├── Tarea
└── Inventario

INTERFACES
│
└── Interactuable

DAO
│
├── EntidadDAO
└── Implementación correspondiente

SERVICE
│
└── EntidadService

EXCEPCIONES
│
├── InteraccionInvalidaException
├── TareaNoDisponibleException
├── ObjetoNoDisponibleException

├── ClienteNoDisponibleException
└── OpcionInvalidaException

MAIN
│
└── Main

2. **Las clases del modelo: cuál es la clase base, cuáles heredan, y por qué eligieron esa jerarquía (qué comparten, qué las diferencia).**

La clase base de nuestro modelo es **Entidad**, que será una clase abstracta. Esta clase tendrá los atributos que comparten los elementos principales del juego, como id y nombre.

Entidad heredan 3 subclases, que son las siguientes:

Personaje
↓
Objeto
↓
Tarea

Personaje hereda a Jugador y NPCs.

NPC hereda a Cliente.

Objeto no hereda a nada, ya que representa los objetos que pueden encontrar y utilizarse durante el juego.

Tarea tampoco hereda a nada, ya que representa las diferentes actividades que el jugador debe realizar.

Existe también la clase Inventario, que se encarga de almacenar los objetos que posee el jugador.

Elegimos esta jerarquía porque las clases principales representan diferentes elementos del juego y comparten características generales, pero independientemente tienen sus propias características y comportamientos. Por ejemplo, un Jugador tiene el día actual, dinero y tareas completadas, mientras que un Cliente tiene un pedido y un NPC puede tener un rol.

3. **Atributos de cada clase (los del padre + los específicos de cada hija).**

Atributos de las clases:

Entidad

- id
- nombre

Personaje

Además de los atributos heredados de Entidad:

- diálogo

Jugador

Además de los atributos heredados de Personaje:

- diaActual
- tareasCompletadas
- dinero

NPC

Además de los atributos heredados de Personaje:

- rol

Cliente

Además de los atributos heredados de NPC:

- pedido

Objeto

Además de los atributos heredados de Entidad:

- tipoObjeto
- utilizable
- cantidad

Tarea

La clase Tarea representará las actividades que el jugador debe completar durante los diferentes días.

- id
- descripción
- tipo
- dia
- completada

Inventario

La clase Inventario permitirá almacenar los objetos que el jugador posee durante la partida.

- objetos
- cantidades

4. **Diseño de la tabla en MySQL (el CREATE TABLE, igual que venimos haciendo, con la columna tipo si van a usar tabla única para las subclases).**

Decidimos utilizar una única tabla, como venimos haciendo desde que empezamos este tema para que sea más simple. (Base de datos se llama Buffet)

```
CREATE TABLE entidades (
id INT PRIMARY KEY AUTO_INCREMENT,
nombre VARCHAR(50) NOT NULL,
tipo VARCHAR(30) NOT NULL,
dia_actual INT,
tareas_completadas INT,
dinero INT,
dialogo VARCHAR(255),
rol VARCHAR(50),
pedido VARCHAR(100),
tipo_objeto VARCHAR(50),
utilizable BOOLEAN,
cantidad INT,
descripcion VARCHAR(255),
tipo_tarea VARCHAR(50),
completada BOOLEAN,
dia INT
);
```

El "tipo" sirve para saber si es un jugador, cliente, empleado, objeto o tarea.

5. Lista de reglas de negocio que va a validar el service (mínimo 5), y qué excepción dispara cada una.

- Si la tarea no está disponible: **TareaNoDisponibleException**
- Si el objeto no está disponible: **ObjetoNoDisponibleException**
- Si no hay ningún cliente disponible: **ClienteNoDisponibleException**
- Si se selecciona una opción que no existe: **OpcionInvalidaException**
- Si no se puede interactuar: **InteraccionInvalidaException**

Qué interfaz van a definir y en qué método del service la van a usar de forma polimórfica.

Interfaz que vamos a usar: Interactuable

Método en que la vamos a usar: void interactuar();

La interfaz va a ser usada en clases que puedan tener interacción. (Ej: el objeto, el cliente o la tarea)

7. **Métodos previstos del service (nombres, no implementación)**

Métodos para las entidades

crearEntidad()
buscarEntidad()
listarEntidades()
eliminarEntidad()

Métodos para la jugabilidad

realizarTarea()
atenderCliente()
esperarCliente()
abrirInventario()
usarObjeto()
avanzarDia()

Métodos para los clientes

generarCliente()
mostrarPedido()
entregarPedido()

Métodos para las tareas

generarTarea()
mostrarTarea()
completarTarea()

Métodos para el inventario

mostrarInventario()
agregarObjeto()
eliminarObjeto()

Método polimórfico

interactuar(Interactuable elemento)

El Service podrá recibir diferentes tipos de objetos interactuables y ejecutar su comportamiento correspondiente sin depender de una subclase específica.

Fin:D

ACTUALIZACIONES DE LA EJERCITACIÓN 16

Para esta segunda parte del trabajo, nos dedicamos a hacer la parte visual en Swing. Implementando los elementos gráficos pidiéndole ayuda a la IA.

Nuestros PROMPTS:

En el primero, le enviamos adjuntado el ZIP de Buffet y el PDF Ejercitación 16 - Videojuego GUI: "¡Acá tenes el zip de Buffet! Necesito que, tomes en cuenta la consigna, me expliques lo que haces y sigas paso a paso lo que me explicaste: Crear una ventana gráfica, hacer un formulario, hacer una tabla, mostrar los errores en ventanas, y yo lo subo al github. Necesito que la interfaz NO sea como una consola, sino que sea similar a una aplicación o página para que se ejecute de forma más entendible. Necesito que se pueda jugar solo en PC y que no cambies absolutamente nada de sus funciones, en si, toma de referencia todos nuestros archivos compartidos y tu explicación y hace lo que te dice el primer PDF que te envie teniendo en cuenta el ZIP de Buffet. Hacelo para JAVAFX o SWING, el que quieras"

Segundo prompt: "Quiero que lo hagas más interactivo y visual, no un juego solo de texto y botones. Que tenga un personaje que pueda mover e interactuar con el cliente o con la tarea físicamente, en un plano 2d, tipo juego flash."

Tercer prompt, donde le adjuntamos el resultado que nos dió lo del primer buffet: "Descomprime este zip. Y con todo esto, agregale mas tareas, tipo mas clientes (que sean 5 clientes), limpiar, etc. Agregale 6 tareas mas! Para que sea mucho mas

interactivo: NO CAMBIES el estilo! Nos gusta así, solo añadile más tareas e interacciones tanto en el código como en la parte visual!”

Cuarto prompt: “Hola Claude. Necesito que me arregles la interfaz para que se vea mas lindo visualmente. Más como un juego real, más detallado y profesional, priorizando que todo esté en su lugar, una estética indie. El sistema del juego esta bien hecho, centra todo en mejorar el apartado gráfico, agregando detalles para que se vea un juego natural y con una estética similar a "katana zero"”

Quinto prompt: “Hola Claude. Gestiona los tokens adecuadamente para no dejar las modificaciones pendientes. Necesito que me arregles la interfaz para que se vea mas lindo visualmente. Más como un juego real, más detallado y profesional, priorizando que todo esté en su lugar, una estética indie. centra todo en mejorar el apartado gráfico, agregando detalles para que se vea un juego natural y con una estética bien pulida. Mejora las mecánicas y funciones del juego, mezclando cosas como tener que llevar físicamente los productos al cliente en lugar de hacerlo con botón únicamente, llevar objetos para reparar las cosas o donde haya una tarea, que haya mas salas a las que puedas cruzar yendo al borde de la pantalla pasando por una puerta, como el almacén donde estén las cosas de limpieza, o la cocina, etc, hace muchas mejoras en este aspecto, reconstruyendo y mejorando las mecánicas por completo, haciéndolo más entretenido, más duradero, y escalable, también agregando la función de los días, pasando el dia, hace que si ya terminaste la jornada vengan más clientes al dia siguiente, tambien que el tener la cordura baja tenga consecuencias, como que cause que si pasas de dia con baja cordura pierdas automáticamente, sin dejar de lado lo grafico, haciendo todo esto estéticamente impecable, pero sin formas robotizadas”

Sexto prompt: “Al final si abria bien solo que como la pantalla principal donde seleccionas la partida sigue igual pense que no lo habias cambiado, necesito que cambies esa pantalla acorde a todo lo demás, que la letra pixel que es pequena sea un poco mas entendible, y que revises bien las posiciones de cada sprite y como están hechos para ponerlos adecuadamente, la parrilla no se ve correctamente, la parte del mostrador no tiene mucho sentido ya que los clientes se ponen en la pared y parece que caminan por esta, revisa todas esas cosas que no encajen. Tambien arregla la resolución para una pantalla de 1920x1080, y que funcionen correctamente las puertas”

Todo lo demás pedido está dentro del README.md en Buffet en GitHub. (En buffet-TheGame.zip)